Mark	11/11

Team name:	B9		
Homework number:	Homework 11		
Due date:	02/12/2025		
Contribution:	No	Partial	Full
Pica Mirko			✅
Prendin Christian			✅
Pignataro Gabriele			✅
Moazenkakhki Mostafa			✅
Yarrabbotula Sunandha			✅
Notes:			

Project name			
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			✅

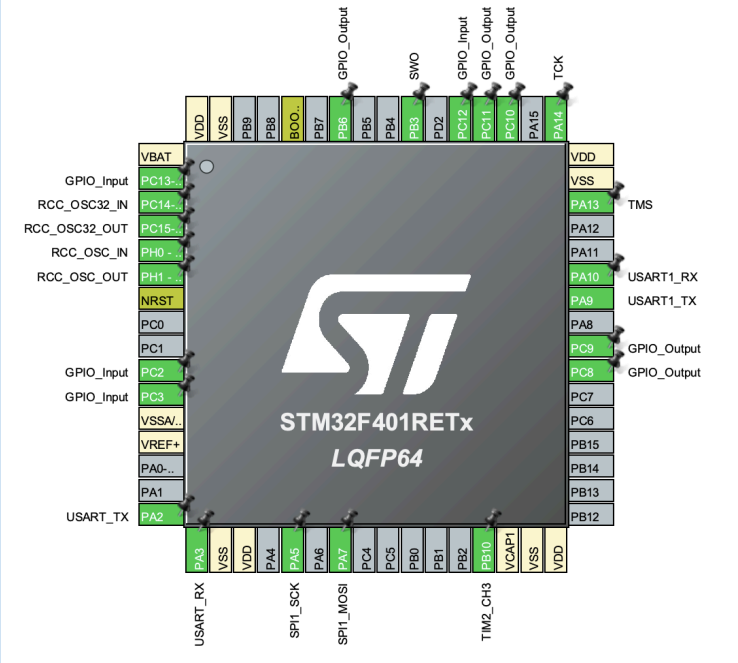
Part 1. Realize the IR Project 2 or create your own project with other peripherals but still using the IR communication.

We described only how to do project 3 as project 2 is the same but the logic is separated into two boards.

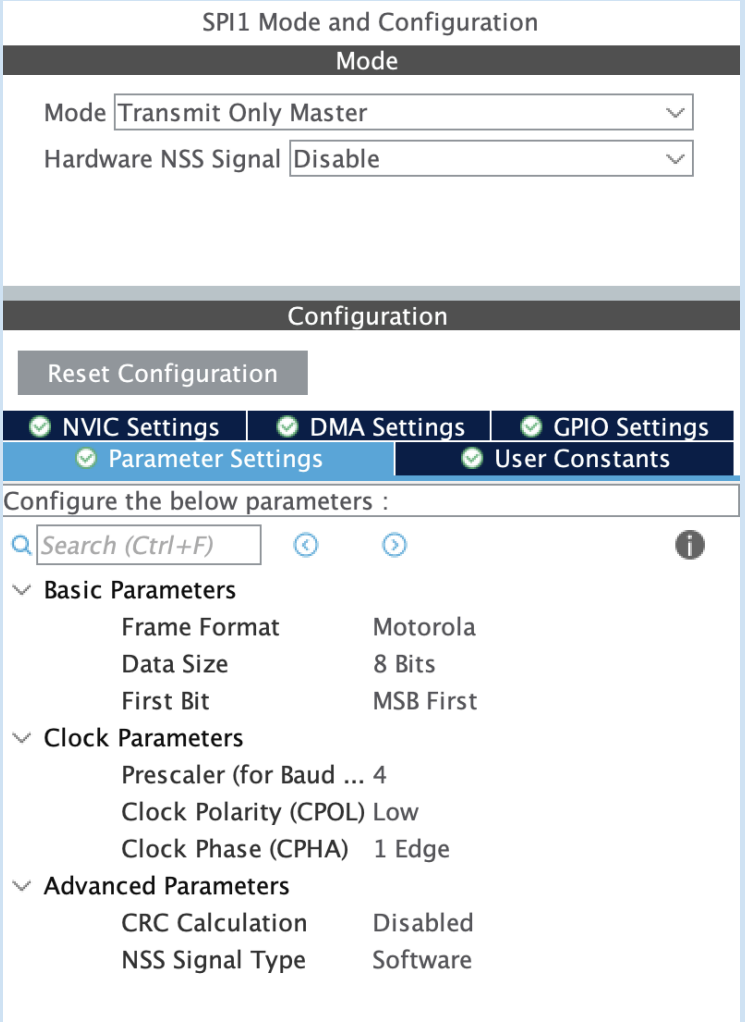
Part 2. Complete Project 3 (IR transmission and reception on the same board).

First we have to prepare all our peripherals and other settings in the .ioc file.

We configure the pins for the keyboard (as done in previous homework), the SPI to communicate with the led matrix (as done in previous homework), the UART1 for the IR receiver on PA10 and PB10 for the IR led with PWM generation (as specified in the slide).



In the connectivity tab we configure the SPI for the LED matrix. We enable and configure it in Transmit Only Master mode. In the Parameter Settings menu, the only parameter that was modified was the prescaler, which was set to 4. We also enable the DMA and the SPI interrupt.



Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

DMA Request	Stream	Direction	Priority
SPI1_TX	DMA2 Stream 3	Memory To Peripheral	Low

Add Delete

DMA Request Settings

		Peripheral	Memory
Mode	Normal	Increment Address	☐	☑
Use Fifo ☐ Threshold		Data Width	Byte	Byte
		Burst Size		

Reset Configuration

NVIC Settings | DMA Settings | GPIO Settings | Parameter Settings | User Constants

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Pri
SPI1 global interrupt	☑	0	0
DMA2 stream3 global interrupt	☑	0	0

In the connectivity section we also configure USART1 to receive via IR. We just set the baud rate to 2400 and enable the interrupt.

USART1 Mode and Configuration

Mode

Mode Asynchronous

Hardware Flow Control (RS232) Disable

Configuration

Reset Configuration

NVIC Settings | DMA Settings | GPIO Settings | Parameter Settings | User Constants

Configure the below parameters :

Search (Ctrl+F)

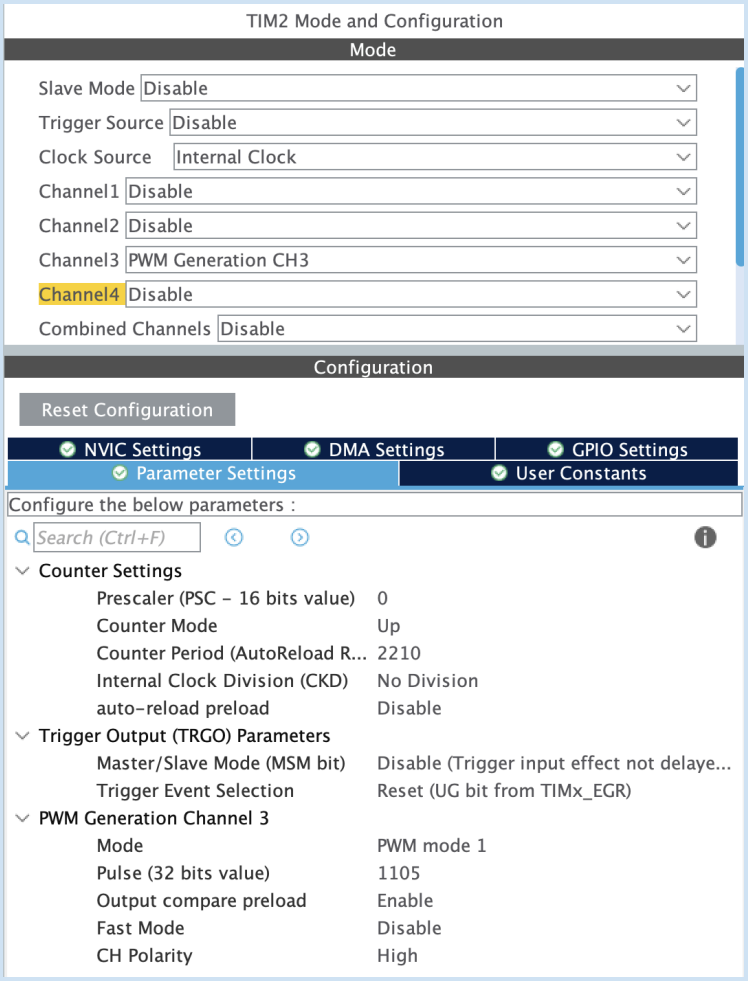
Basic Parameters	
Baud Rate	2400 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1
Advanced Parameters	
Data Direction	Receive and Transmit
Over Sampling	16 Samples

NVIC Settings | DMA Settings | GPIO Settings | Parameter Settings | User Constants

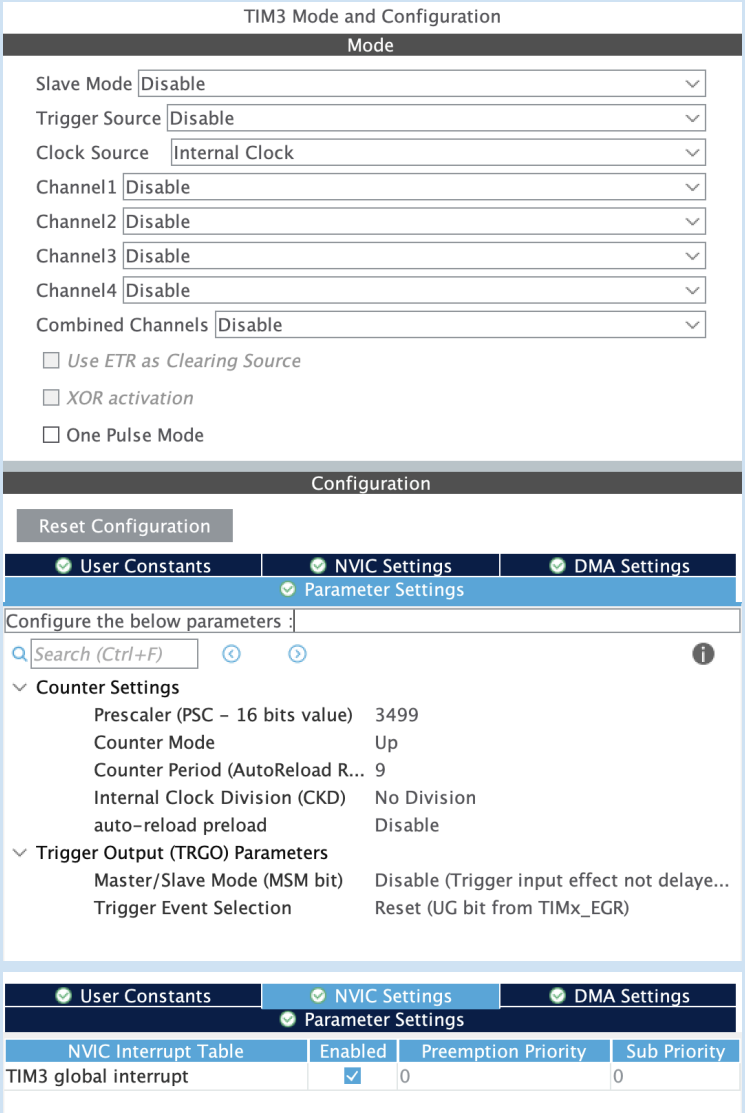
NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
USART1 global interrupt	☑	0	0

We now need to configure all the timers, for PWM generation and for led matrix and keyboard.

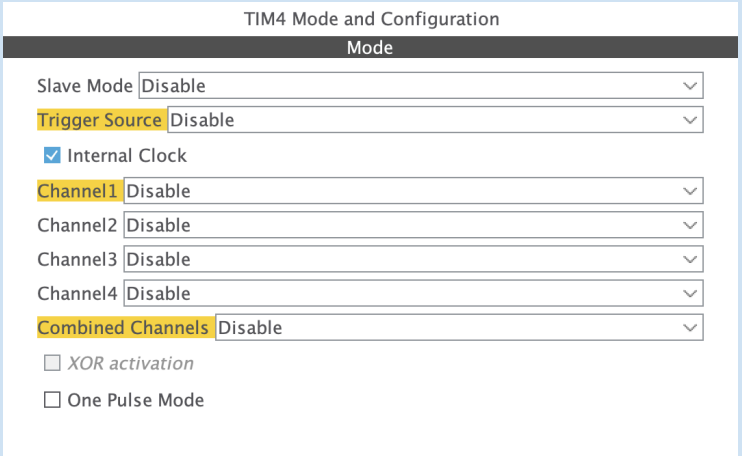
In TIM2 we setup the PWM generation for the IR led. We set the counter period to 2210 to get as close as possible to 38 kHz and the pulse to 1105 to get 50% duty cycle.

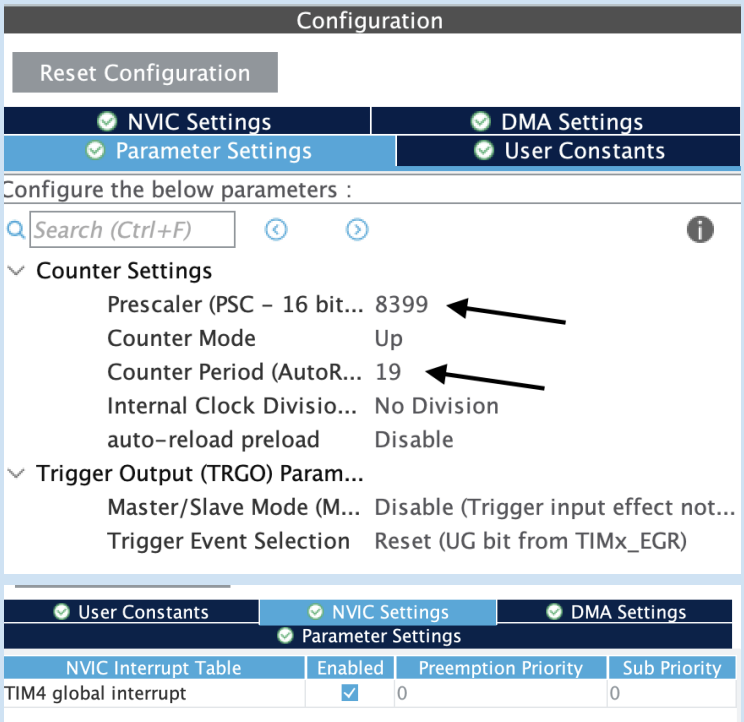


We use TIM3 to get the desired transmission baud rate of 2400. So we set the prescaler to 3499 and the counter to 9. We enable the interrupt.

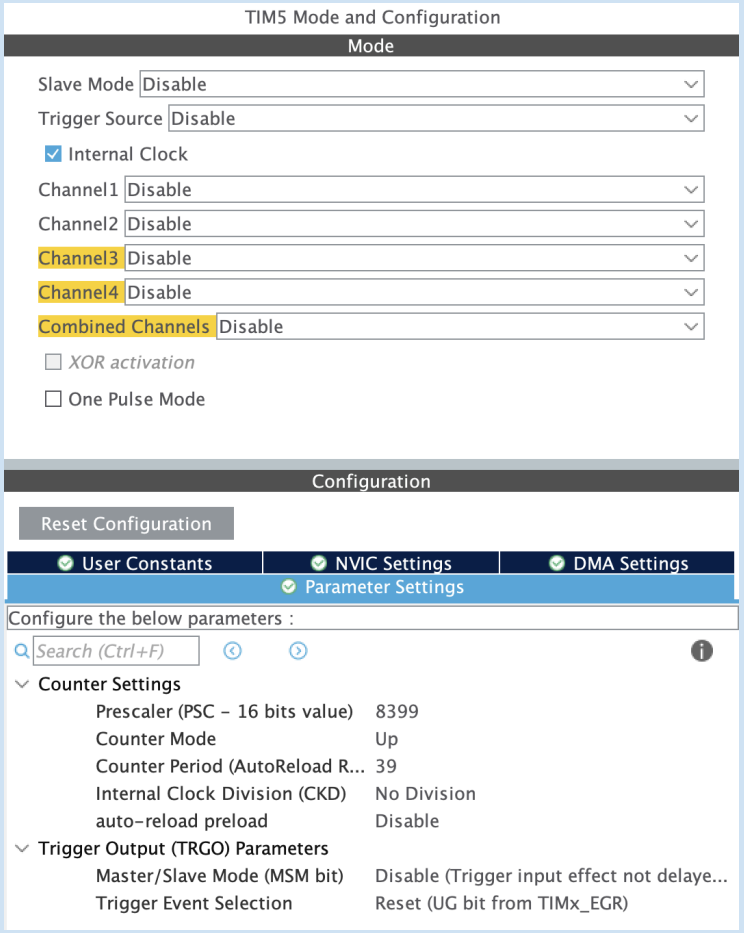


TIM4 is triggered every 2 ms to check keyboard presses as done in previous homework.





TIM5 is triggered every 4 ms to update the led matrix column by column as done in previous homework.



User Constants | NVIC Settings | DMA Settings

Parameter Settings

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
TIM5 global interrupt	☑	0	0

We can now go into the “main.c” file and start coding. In the main function we just need to start listening on UART1, start checking on the keyboard and start updating the led matrix.

```
/* USER CODE BEGIN 2 */
// IR Communication
HAL_UART_Receive_IT(&huart1, &rx_buffer, 1);

// Keyboard
HAL_GPIO_WritePin(GPIOC, COLUMN_PINS[current_column], GPIO_PIN_SET);
HAL_TIM_Base_Start_IT(&htim4);

// LED Matrix
HAL_TIM_Base_Start_IT(&htim5);
```

We need to define some variables to help us in the callbacks. The most important are:

- tx_state that will manage ir communication making sure to send start and stop bits correctly;
- KEY_MAP that we configured to send the index corresponding to the array of the led matrix;
- rows the array containing the values to send to the led matrix to obtain the correct character.

```
/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
#define COL 5
/* USER CODE END PD */
```

```c
/* USER CODE BEGIN PV */
// IR
uint8_t rx_buffer;
uint8_t tx_buffer;

int tx_state;
int bit_index;

// KEYBOARD
const uint16_t ROW_PINS[4] = {GPIO_PIN_3, GPIO_PIN_2, GPIO_PIN_13, GPIO_PIN_12};
const uint16_t COLUMN_PINS[4] = {GPIO_PIN_11, GPIO_PIN_10, GPIO_PIN_9, GPIO_PIN_8};

const char KEY_MAP[4][4] = {
    {0x0F, 0x0E, 0x0D, 0x0C}, // f, e, d, c -> 15, 14, 13, 12
    {0x0B, 0x0A, 0x09, 0x08}, // b, a, 9, 8 -> 11, 10, 9, 8
    {0x07, 0x06, 0x05, 0x04}, // 7, 6, 5, 4
    {0x03, 0x02, 0x01, 0x00}  // 3, 2, 1, 0
};

volatile int current_column_key = 0;
volatile uint8_t key_counter[4][4] = {0};

// LED MATRIX
const uint8_t columns[COL] = {16, 8, 4, 2, 1};
const uint8_t rows[16][COL] = {
        {127, 65, 65, 65, 127}, // 0
        {17, 33, 127, 1, 1}, // 1
        {35, 69, 73, 81, 33}, // 2
        {65, 73, 73, 73, 54}, // 3
        {120, 8, 8, 127, 8}, // 4
        {121, 73, 73, 73, 70}, // 5
        {63, 73, 73, 73, 70}, // 6
        {64, 64, 67, 76, 112}, // 7
        {54, 73, 73, 73, 54}, // 8
        {48, 72, 72, 72, 127}, // 9
        {31, 36, 68, 36, 31}, // A
        {127, 73, 73, 73, 54}, // B
        {62, 65, 65, 65, 34}, // C
        {127, 65, 65, 65, 62}, // D
        {127, 73, 73, 73, 65}, // E
        {127, 72, 72, 72, 64} // F
};

volatile int current_column = 0;
volatile int current_symbol = 0;

uint8_t spi_tx_buffer[2];
/* USER CODE END PV */
```

We wrote a helper function to start the IR transmission. We check if we are already transmitting, if not we start.

In the receive callback, as suggested in the slide we save the value in a buffer and restart the receive immediately.

```c
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
// IR transmitting
void sendByte(uint8_t data) {
    if(htim3.Instance->CR1 & TIM_CR1_CEN) return;

    tx_buffer = data;
    tx_state = 1;
    __HAL_TIM_SET_COUNTER(&htim3, 0);
    HAL_TIM_Base_Start_IT(&htim3);
}

// IR Receiving
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
  if (huart->Instance == USART1) {
      current_symbol = rx_buffer;
      HAL_UART_Receive_IT(&huart1, &rx_buffer, 1);
  }
}
```

The timer elapsed callback is the most complicated function because it handles all the different timers.

We managed the IR transmission protocol with a switch case to correctly handle start and stop bits. In the data phase we used a shift with an and to check bit by bit if it was a 0 or 1.

The keyboard and the led matrix callbacks are the same as previous homeworks.

```c
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    // IR Communication
    if (htim->Instance == TIM3) {
        switch (tx_state) {
            case 1:
                HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
                tx_state = 2;
                bit_index = 0;
                break;
            case 2:
                if (bit_index < 8) {
                    if (tx_buffer & (1 << bit_index)) {
                        HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                    } else {
                        HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
                    }
                    bit_index++;
                } else {
                    tx_state = 3;
                }
                break;
            case 3:
                HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_3);
                tx_state = 4;
                break;
            case 4:
                HAL_TIM_Base_Stop_IT(&htim3);
        }
    }
    // Keyboard
    if (htim->Instance == TIM4) {
        for (int i = 0; i < 4; i ++) {
            if (!HAL_GPIO_ReadPin(GPIOC, ROW_PINS[i])) {
                if (key_counter[i][current_column_key] < 255) {
                    key_counter[i][current_column_key]++;
                    if(key_counter[i][current_column_key] == 10)
                        sendByte(KEY_MAP[i][current_column_key]);
                }
            } else {
                key_counter[i][current_column_key] = 0;
            }
        }

        HAL_GPIO_WritePin(GPIOC, COLUMN_PINS[current_column_key], GPIO_PIN_RESET);
        current_column_key = (current_column_key + 1) % 4;
        HAL_GPIO_WritePin(GPIOC, COLUMN_PINS[current_column_key], GPIO_PIN_SET);
    }
    // LED Matrix
    if (htim->Instance == TIM5) {
      spi_tx_buffer[0] = rows[current_symbol][current_column];
      spi_tx_buffer[1] = columns[current_column];
      HAL_SPI_Transmit_DMA(&hspi1, spi_tx_buffer, 2);
    }
}
```

There is also the SPI complete callback for the LED matrix (as seen in previous homework).

```c
// SPI Communication with LED matrix
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi){
    if (hspi->Instance == SPI1) {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);

        current_column = (current_column + 1) % COL;
    }
}
/* USER CODE END 0 */
```

We have tested it and it works even on the same board, to be honest it's difficult to stop the board from self communicating with itself.
Maybe introducing a software change to block the receiving when we press a key would be a nice future add-on.

Professor comments:	